

KL-Regularized Diffusion Policy Steering under Sparse Rewards

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

Abstract—Diffusion policies trained via behavioral cloning (BC) are effective for robot manipulation, but fine-tuning them online with reinforcement learning (RL) remains difficult. The Diffusion Steering via Reinforcement Learning (DSRL) framework addresses this by training a secondary RL policy over the initial noise input of a frozen diffusion model, enabling sample-efficient policy improvement without modifying model weights. DSRL operates under sparse rewards by design, yet training stability remains an open question: the noise policy may over-specialize toward early successful trajectories when reward signals are sparse, leading to high variance across runs. We propose augmenting DSRL with a KL divergence penalty toward the Gaussian prior to mitigate this, and evaluate its effect under both sparse and dense reward settings on the Robomimic benchmark. We outline the research questions, experimental design, and evaluation strategy for this project.

Keywords—diffusion policy; reinforcement learning; KL divergence; latent noise space; sparse reward; robot manipulation

I. PROBLEM STATEMENT

A. Overall Goal

The overall goal of this project is to investigate whether KL divergence regularization on the DSRL latent noise policy improves training stability and sample efficiency in sparse-reward robot manipulation tasks, and to characterize the sensitivity of this effect to the regularization strength β .

B. Problem Domain

The problem domain is robot manipulation in simulation, evaluated on the Robomimic benchmark [12]. The agent observes robot proprioception (joint positions, velocities, and end-effector pose) and task-relevant object state (object position and orientation), following Robomimic’s standard low-dimensional observation space. It must produce continuous 7-DoF end-effector actions. The state space S is continuous and low-to-medium dimensional; the action space A is continuous (7-dimensional). The environment dynamics are stochastic due to contact physics and simulation variability.

Reward structure is a key variable of interest. DSRL uses a binary sparse reward by default on Robomimic: $r = +1$ if the task success predicate is satisfied, $r = 0$ otherwise. Sparse rewards are realistic—real robot deployments rarely provide continuous shaping signals—but are known to amplify exploration and stability challenges in RL. To isolate the effect of reward structure, we evaluate both sparse and dense reward settings, using a 2×2 design: $\{\text{Vanilla DSRL, KL-DSRL}\} \times \{\text{sparse reward, dense reward}\}$.

In the DSRL framework, the RL policy acts in the latent

noise space W rather than the original action space A . It selects the initial noise vector w at each step; the frozen diffusion policy denoises w into an action. The noise space is continuous and has the same dimensionality as the diffusion policy’s action representation.

C. What the System is Learning

The system learns a noise policy $\pi_W : S \rightarrow W$ that maps robot observations to noise vectors which, when denoised through the frozen diffusion policy, produce task-completing actions. The policy is parameterized as a Gaussian MLP outputting mean and variance over noise vectors.

The system has learned correctly if: (1) task success rate improves over the BC baseline; (2) learning curves are stable across random seeds; and (3) the policy reaches a target success rate in fewer episodes than vanilla DSRL.

D. Interaction with the Physical World

All experiments are conducted in MuJoCo simulation via the Robomimic framework. The robot is a 7-DOF Franka Panda arm. There is no direct human interaction during RL training; human demonstrations are used only for BC pretraining, provided by Robomimic’s standard dataset (200 demonstrations per task).

E. Why This Problem is Interesting

DSRL is compelling because it avoids expensive backpropagation through the denoising chain and requires only black-box access to the base policy. However, even under its default sparse reward setting, the noise policy may over-specialize toward latent regions associated with early successes, leading to high variance across runs and poor sample efficiency. This risk is inherent to sparse reward RL: when reward signals are rare, early successful trajectories can dominate the replay buffer and cause the policy to collapse to a narrow region of noise space. Understanding whether a simple KL penalty can mitigate this is both practically relevant and theoretically interesting, given the structural differences between the noise space and the policy spaces studied in prior regularization work.

II. RELATED WORK

A. Diffusion Policies for Behavioral Cloning

Chi et al. [1] introduced Diffusion Policy, representing a robot’s visuomotor policy as a conditional denoising diffusion process. At inference time, the policy iteratively denoises a Gaussian noise sample into an action conditioned on the current observation. This formulation handles multimodal action distributions well and outperformed prior methods on 15 manipulation tasks. Training is notably stable compared to energy-based alternatives [10].

Wang et al. [2] extended diffusion models to offline RL with Diffusion-QL, jointly training a Q-function alongside the diffusion policy. The Q-value gradient steers generated actions toward high-value regions while the diffusion prior acts as an implicit behavioral constraint. This achieves strong results on the D4RL benchmark. Both works are ultimately bounded by their training data—the central limitation this project addresses. We include Diffusion-QL as an optional reference baseline to contextualize the performance achievable from offline diffusion-based RL methods; since it operates in a fundamentally different setting (offline vs. online), it serves as a ceiling reference rather than a direct competitor.

B. RL Fine-Tuning of Diffusion Policies

Ren et al. [4] introduced DPPO, applying PPO to diffusion policy fine-tuning by treating each denoising step as an inner action. DPPO achieves state-of-the-art results on Robomimic and FurnitureBench, but requires direct weight access and is computationally expensive due to backpropagation through the full denoising chain.

Wagenmaker et al. [3] proposed DSRL, which treats the initial noise vector w as the RL action. A small MLP selects w ; the frozen denoiser converts it to an action; the environment returns a reward. The RL policy is trained with RLPD [5], an off-policy actor-critic that is much cheaper than backpropagating through the denoiser. DSRL achieves strong results on Robomimic and Gym benchmarks, and improves a real robot task from 20% to 90% success in fewer than 50 episodes. On Robomimic, DSRL uses a binary 0-1 success reward by default. While results are promising, training stability and sample efficiency under this sparse reward regime remain open questions—particularly whether the noise policy is prone to over-specialization across seeds. This is the central question this project investigates.

The shared weakness of both DPPO and DSRL under sparse rewards is the absence of an explicit diversity constraint. Without one, the RL component may over-specialize to locally rewarding behaviors early in training, leading to high variance across seeds.

C. KL Divergence Regularization in RL

TRPO and PPO [7] introduced KL divergence as a constraint on consecutive policy updates to prevent destabilizing gradient steps. More relevant to this project is its use in RLHF for language models, where the reward is augmented as $\hat{r} = r - \beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ [9]. The penalty prevents the policy from exploiting weaknesses in the reward signal while forgetting pretraining.

A key question is why KL regularization is preferable to alternatives. An entropy bonus encourages the policy to remain internally diverse but places no constraint on how far it drifts from the reference prior. A conservative update constrains step-by-step changes but does not anchor the policy to the original prior over time. KL regularization directly targets the failure mode of interest: it penalizes the

noise policy for drifting far from $N(0, I)$, the prior the frozen diffusion model was designed to invert. The KL penalty regularizes each state-conditioned noise distribution independently—it does not force distributions at different states to coincide. Distinct output behaviors can still emerge across states so long as noise distributions remain within the support of the prior.

Because the noise policy is a Gaussian MLP, $D_{\text{KL}}(\pi_w \parallel N(0, I))$ has a closed-form expression: $\frac{1}{2} \Sigma(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$, requiring no Monte Carlo estimation. This makes the penalty straightforward to implement on top of the existing DSRL objective [5].

III. RESEARCH QUESTIONS

RQ1. Does a KL divergence penalty on the DSRL noise policy improve training stability under sparse rewards, measured as reduced variance in learning curves across random seeds?

RQ2. Does the penalty improve sample efficiency—does the regularized variant reach a target success rate in fewer environment interactions than vanilla DSRL?

RQ3. How sensitive is performance to the penalty coefficient β ? Is there a range of β that reliably improves upon vanilla DSRL, or does the benefit disappear as β grows?

These questions have not been directly addressed in prior work. The closest analogy is KL regularization in RLHF [9], but that setting differs in important ways: the noise space is low-dimensional and Gaussian, and the frozen denoiser already provides some implicit regularization. Whether an additional explicit penalty is helpful, redundant, or harmful under sparse-reward manipulation is an open empirical question. We focus on KL regularization as the primary intervention; comparison with alternative regularization strategies such as entropy bonus is treated as an optional extension if experimental resources permit.

IV. HYPOTHESIZED FINDINGS

H1. A moderate KL penalty will reduce variance across seeds in sparse-reward tasks by preventing the noise policy from concentrating too quickly on a narrow region of noise space, reducing sensitivity to early random rewards.

H2. Sample efficiency will improve at moderate β but degrade at large β . When the penalty is too strong, the noise policy is forced to remain close to $N(0, I)$, effectively reverting to BC-level behavior and eliminating the RL benefit.

H3. The effective range of β will be constrained by the implicit regularization already imposed by the diffusion prior. Because the frozen denoiser maps noise vectors to actions in a many-to-one fashion, small perturbations in w do not always produce proportionally large changes in output behavior. We therefore predict that small values of β (e.g., 0.01–0.1) will be sufficient to improve stability, and that performance will degrade as β grows beyond this range—forcing the noise policy to stay so close to $N(0, I)$ that the RL signal is effectively suppressed, reverting

toward BC-level behavior. The β sweep (0.01, 0.1, 0.5, 1.0) is designed to directly test this prediction.

We note that the effectiveness of KL regularization may depend on task difficulty, reward sparsity, and action-space complexity. Accordingly, we evaluate across tasks of varying manipulation difficulty to assess the sensitivity of these hypotheses to task characteristics. For each hypothesis, we define concrete criteria. H1 is supported if KL-DSRL under sparse reward shows lower standard deviation of success rate across seeds than vanilla DSRL at convergence; it is refuted if the variance difference is negligible or reversed. H2 is supported if KL-DSRL reaches the same final success rate as vanilla DSRL in fewer timesteps; it is refuted if convergence speed is similar or slower. H3 is supported if the β sweep shows a non-monotonic pattern—small β improving over vanilla DSRL and large β degrading below it; it is refuted if performance is flat across β values. If all hypotheses are refuted, the null result is itself informative: it would suggest that DSRL’s implicit regularization from the diffusion prior is already sufficient, and that explicit KL penalty adds no benefit in this setting. This would be a meaningful negative result about the design space of noise-space RL methods.

V. APPROACH

A. Algorithms

The primary comparison is between two variants. Vanilla DSRL (baseline): the DSRL-NA variant from Wagenmaker et al. [3], using RLPD [5] as the off-policy base. This trains a Q-function on the original action space, distills it into the noise space, and trains the noise policy via SAC-style updates [8].

KL-DSRL (proposed): identical to vanilla DSRL, with one modification to the actor loss:

$$L_{\text{actor}} = L_{\text{DSRL}} - \beta \cdot D_{\text{KL}}(\pi_W(\cdot|s) || N(0, I))$$

This modification requires approximately five lines of code in the DSRL actor update and introduces no additional computational overhead beyond the closed-form KL computation.

Algorithm 1 describes Vanilla DSRL; Algorithm 2 describes KL-DSRL, with the single modified line shown in bold.

```

Algorithm 1: Vanilla DSRL
Input: frozen diffusion policy  $\pi_{\text{dp}}$ , reward function  $r$ 
Init: noise policy  $\pi_W$  (Gaussian MLP), replay buffer  $B$ 
for each training step do
  observe state  $s$ 
  sample  $w \sim \pi_W(s)$  // noise vector
  compute  $a = \pi_{\text{dp}}(s, w)$  // frozen, no gradient
  execute  $a$ , observe reward  $r$ , next state  $s'$ 
  store  $(s, w, r, s')$  in  $B$ 
  sample batch from  $B$ 
  update critic  $Q$  toward Bellman target
  update actor:  $L = L_{\text{DSRL}}$ 
end for

```

```

Algorithm 2: KL-DSRL (proposed)
Input: frozen diffusion policy  $\pi_{\text{dp}}$ , reward function  $r$ , KL coefficient  $\beta$ 
Init: noise policy  $\pi_W$  (Gaussian MLP), replay buffer  $B$ 
for each training step do
  observe state  $s$ 
  sample  $w \sim \pi_W(s)$ 
  compute  $a = \pi_{\text{dp}}(s, w)$ 
  execute  $a$ , observe reward  $r$ , next state  $s'$ 
  store  $(s, w, r, s')$  in  $B$ 
  sample batch from  $B$ 
  update critic  $Q$  toward Bellman target
  update actor:  $L = L_{\text{DSRL}} - \beta \cdot \text{KL}(\pi_W(\cdot|s) || N(0, I))$ 
  where  $\text{KL} = 1/2 \cdot \sum(\mu^2 + \sigma^2 - \log\sigma^2 - 1)$  // closed form
end for

```

B. Environment and Task Selection

We use the Robomimic benchmark [12] with two tasks spanning distinct difficulty levels: Can (pick-and-place a soda can into a bin) and Square (insert a square nut onto a peg). These tasks span qualitatively distinct manipulation primitives—single-stage pick-and-place and precision insertion—and differ substantially in difficulty, providing a natural gradient for evaluating robustness. This difficulty gradient allows us to evaluate whether the effectiveness of KL regularization depends on task complexity and action-space precision requirements, directly addressing the sensitivity of our hypotheses to task characteristics.

We evaluate under both sparse and dense reward conditions. For the sparse setting, we use DSRL’s default binary reward: $r = +1$ on task completion, $r = 0$ otherwise. For the dense setting, we use Robomimic’s shaped reward signal, which provides intermediate feedback proportional to task progress. This 2×2 design allows us to isolate whether KL regularization provides benefit specifically under sparse rewards, or also extends to the dense reward setting. DSRL provides pretrained diffusion policy checkpoints for both tasks, eliminating the need for BC pretraining.

C. Expected Output and Data Collection

For each task and algorithm, the system produces: (1) a trained noise policy π_W ; (2) success rate learning curves across seeds; and (3) final success rates at varying β values. No external data collection is required; Robomimic provides 200 human teleoperated demonstrations per task for BC pretraining, and RL data is collected online within the MuJoCo simulation.

D. Simplest First Test

Train vanilla DSRL on Robomimic-Can with sparse reward for an initial pilot budget of several hundred episodes across multiple random seeds, comparable to the training horizon used in prior DSRL experiments [3]. We hypothesize that learning remains possible but exhibits higher variance than in the dense-reward setting, establishing the failure mode we aim to mitigate. If Can-sparse proves too easy, we will reduce the number of demonstrations to weaken the BC initialization, or switch to

the harder Square task. Before running full experiments, we verify the KL-DSRL implementation with three sanity checks. First, setting $\beta = 0$ must reproduce identical learning curves to vanilla DSRL; any divergence indicates a bug in the actor loss. Second, at initialization, $D_{\text{KL}}(\pi_W \parallel N(0, I))$ should be near zero, since π_W is initialized close to the prior; a large initial KL suggests incorrect parameterization. Third, we hand-compute the KL for a single batch using the closed-form expression $\frac{1}{2}\Sigma(\mu^2 + \sigma^2 - \log\sigma^2 - 1)$ and verify it matches the code output numerically.

E. Computing Resources

All experiments will run on Yale's Bouchet HPC cluster. Training is expected to require 2–4 GPU-hours per run on Robomimic-Can and 4–8 GPU-hours per run on Square; all runs are parallelized across available GPUs. The β hyperparameter is selected using the Can-sparse condition as a tuning set, sweeping $\beta \in \{0.01, 0.1, 0.5, 1.0\}$ and choosing the value that maximizes final success rate; the selected β is then fixed for the Square condition to assess generalization. All training metrics are logged every 10,000 timesteps and saved as CSV files, one per seed per condition, named by condition and seed index (e.g., `kl-dsrl_sparse_seed0.csv`).

VI. EVALUATION

A. Evaluating Research Questions

RQ1 (stability) and RQ2 (sample efficiency): Run vanilla DSRL and KL-DSRL on both tasks, 3 seeds each. Compute the mean and standard deviation of success rate across seeds at each training step. A reduction in variance for KL-DSRL supports H1. We additionally compare how quickly each variant reaches a target success rate as a measure of sample efficiency, supporting H2. At evaluation time, success rate is measured by rolling out the policy stochastically: a noise vector is sampled from the learned distribution $\pi_W(\cdot|s)$ and passed through the frozen diffusion model to generate an action. We use stochastic rather than deterministic (mean-action) inference because mean-action evaluation would suppress the distributional diversity that KL regularization is designed to preserve, making it difficult to assess the regularization's effect on the noise policy. All metrics are recorded every 10,000 timesteps. Learning curves are computed from dedicated evaluation rollouts performed every 10,000 timesteps, separate from training data collection, to ensure that reported success rates reflect true policy performance without exploration noise. In addition to success rate, we record the KL divergence value $D_{\text{KL}}(\pi_W \parallel N(0, I))$ throughout training for all KL-DSRL conditions. This serves as a mechanistic indicator: under appropriate β , KL should remain bounded rather than growing unchecked (as in vanilla DSRL) or being suppressed near zero (as under excessive β). Comparing KL trajectories across β values provides direct evidence for or against H3.

RQ3 (β sensitivity): Sweep $\beta \in \{0.01, 0.1, 0.5, 1.0\}$ on the Can task. Plot final success rate and training variance as

a function of β to characterize the useful operating range.

B. Performance Metrics

TABLE I: Evaluation Metrics

Metric	Supports
Success rate mean $\pm$ std across seeds	RQ1, RQ2
Final success rate vs. penalty coefficient β	RQ3
KL divergence value $D_{\text{KL}}(\pi_W \parallel N(0, I))$ during training	Mechanistic analysis

C. Comparisons

Primary: vanilla DSRL vs. KL-DSRL ($\beta = 0.1$) on Can-sparse and Square-sparse, 3 seeds each. Secondary: β sweep on Can-sparse. We additionally include two dense-reward conditions as reference baselines: vanilla DSRL with dense reward, which quantifies the performance gap introduced by sparse rewards alone; and KL-DSRL with dense reward (ablation), which isolates whether KL regularization provides benefit independently of reward sparsity. We optionally include Diffusion-QL [2] as a further reference to contextualize the performance achievable from offline diffusion-based RL; since it cannot benefit from online interaction, it serves as a ceiling reference rather than a direct competitor. To ensure data isolation across conditions: (1) each condition is trained independently with no shared replay buffer or model weights; (2) all conditions use the same three fixed random seeds (0, 1, 2) for reproducibility and fair comparison; and (3) β sweep experiments are run independently from the main experiments, with no data shared between sweep runs. Each run on Robomimic-Can is estimated to require 2–4 GPU-hours and each Square run 4–8 GPU-hours on a single modern GPU; all 33 runs are parallelized across available GPUs on Yale's Bouchet HPC cluster.

D. Planned Figures

We report four placeholder figures below. Each shows labeled axes and a caption; actual results will replace the placeholders after experiments are run.

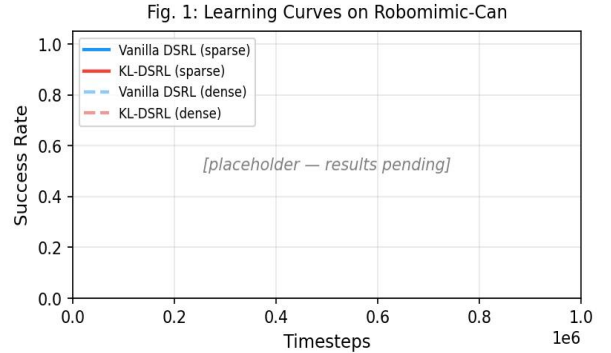


Fig. 1: Learning curves on Robomimic-Can. x-axis: timesteps ($0-1 \times 10^6$); y-axis: success rate ($0-1$). Four conditions shown with mean ± 1 SE. A downward gap between sparse and dense lines reflects reward difficulty; an upward gap between KL-DSRL and

vanilla within each reward type supports H1/H2.

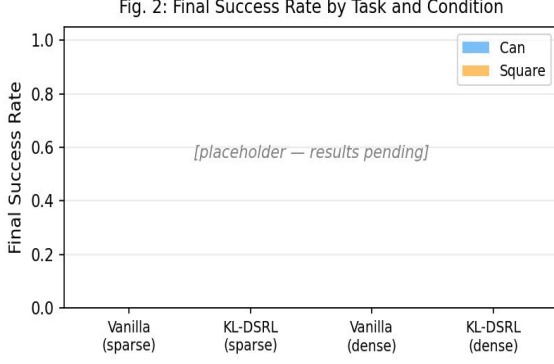


Fig. 2: Final success rate by task and condition. x-axis: four conditions; y-axis: final success rate (0–1). Grouped bars for Can (blue) and Square (orange). Higher KL-DSRL bars than vanilla within each reward type would support H2; consistent pattern across tasks would show generalizability.

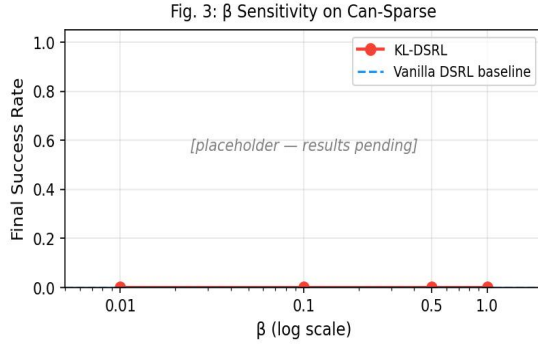


Fig. 3: β sensitivity curve on Can-sparse. x-axis: $\beta \in \{0.01, 0.1, 0.5, 1.0\}$ (log scale); y-axis: final success rate. Dashed line marks vanilla DSRL performance. A non-monotonic curve peaking at small β and falling below baseline at large β supports H3.

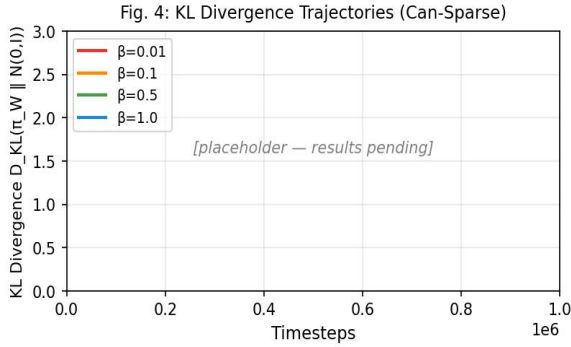


Fig. 4: KL divergence trajectories on Can-sparse. x-axis: timesteps; y-axis: $D_{KL}(\pi_W || N(0, I))$. One line per β value. Bounded KL at small β and near-zero KL at large β would provide mechanistic evidence for H3.

VII. TIMELINE

TABLE II: Project Timeline

Phase	Tasks
Week 1: Setup & baseline	Clone DSRL repo; confirm Robomimic-Can pipeline runs; implement sparse reward wrapper; run vanilla DSRL baseline across multiple seeds

	to establish failure mode
Week 2: Implementation & experiments	Implement KL penalty in actor loss; run pilot to verify training signal; full comparison of vanilla DSRL vs. KL-DSRL on Can and Square; β sweep on Can
Week 3: Analysis & writing	Analyze learning curves; compute variance metrics; produce figures; write and revise final report. Contingency: if Can-sparse too easy, reduce demonstrations or switch to Square; report null results if KL has no effect

REFERENCES

- [1] C. Chi et al., "Diffusion policy: Visuomotor policy learning via action diffusion," in Proc. RSS, 2023.
- [2] Z. Wang, J. J. Hunt, and M. Zhou, "Diffusion policies as an expressive policy class for offline reinforcement learning," in Proc. ICLR, 2023.
- [3] A. Wagenmaker et al., "Steering your diffusion policy with latent space reinforcement learning," in Proc. CoRL, 2025.
- [4] A. Z. Ren et al., "Diffusion policy policy optimization," in Proc. ICLR, 2025.
- [5] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data," in Proc. ICML, 2023.
- [6] M. T. Jackson et al., "Policy-guided diffusion," arXiv:2404.06356, 2024.
- [7] J. Schulman et al., "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
- [8] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic," in Proc. ICML, 2018.
- [9] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Proc. NeurIPS, 2022.
- [10] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in Proc. NeurIPS, 2020.
- [11] B. Liu et al., "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in Proc. NeurIPS, 2023.
- [12] A. Mandlekar et al., "What matters in learning from offline human demonstrations for robot manipulation," in Proc. CoRL, 2022.
- [13] L. Black et al., " π_0 : A vision-language-action flow model for general robot control," arXiv:2410.24164, 2024.
- [14] T. Pearce et al., "Imitating human behaviour with diffusion models," in Proc. ICLR, 2023.